

Rank students for admission by sorting scores using Bubble Sort and display top 10 candidates.

```
import java.util.*;

class StudentRanking {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of students: ");
        int n = sc.nextInt();

        int[] scores = new int[n];

        System.out.println("Enter student scores:");
        for (int i = 0; i < n; i++) {
            scores[i] = sc.nextInt();
        }

        // ◊ Bubble Sort (Descending order)
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (scores[j] < scores[j + 1]) {
                    int temp = scores[j];
                    scores[j] = scores[j + 1];
                    scores[j + 1] = temp;
                }
            }
        }

        // ◊ Display Top 10
        System.out.println("\nTop Candidates:");
        int limit = Math.min(10, n);

        for (int i = 0; i < limit; i++) {
            System.out.println("Rank " + (i + 1) + " : " + scores[i]);
        }

        sc.close();
    }
}
```

Shortlist internship candidates by sorting test scores using Selection Sort and display top 10 students.

```
import java.util.*;

class InternshipSelection {
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);

System.out.print("Enter number of students: ");
int n = sc.nextInt();

int[] scores = new int[n];

System.out.println("Enter test scores:");
for (int i = 0; i < n; i++) {
    scores[i] = sc.nextInt();
}

// ◊ Selection Sort (Descending)
for (int i = 0; i < n - 1; i++) {
    int maxIndex = i;

    for (int j = i + 1; j < n; j++) {
        if (scores[j] > scores[maxIndex]) {
            maxIndex = j;
        }
    }

    // swap
    int temp = scores[i];
    scores[i] = scores[maxIndex];
    scores[maxIndex] = temp;
}

// ◊ Display Top 10
System.out.println("\nTop 10 Candidates:");
int limit = Math.min(10, n);

for (int i = 0; i < limit; i++) {
    System.out.println("Rank " + (i + 1) + " : " + scores[i]);
}

sc.close();
}
}

```

Generate a competitive exam leaderboard by sorting scores using Insertion Sort and display top 10 rank holders.

```

import java.util.*;

class LeaderboardInsertionSort {
    public static void main(String[] args) {

```

```

Scanner sc = new Scanner(System.in);

System.out.print("Enter number of students: ");
int n = sc.nextInt();

int[] scores = new int[n];

System.out.println("Enter exam scores:");
for (int i = 0; i < n; i++) {
    scores[i] = sc.nextInt();
}

// ◊ Insertion Sort (Descending)
for (int i = 1; i < n; i++) {
    int key = scores[i];
    int j = i - 1;

    while (j >= 0 && scores[j] < key) {
        scores[j + 1] = scores[j];
        j--;
    }

    scores[j + 1] = key;
}

// ◊ Display Top 10
System.out.println("\nTop 10 Rank Holders:");
int limit = Math.min(10, n);

for (int i = 0; i < limit; i++) {
    System.out.println("Rank " + (i + 1) + " : " + scores[i]);
}

sc.close();
}
}

```

Identify placement-eligible students by sorting aptitude scores using Bubble Sort and display top performers.

```

import java.util.*;

class PlacementEligibility {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of students: ");
        int n = sc.nextInt();
    }
}

```

```

int[] scores = new int[n];

System.out.println("Enter aptitude scores:");
for (int i = 0; i < n; i++) {
    scores[i] = sc.nextInt();
}

// ◊ Bubble Sort (Descending)
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        if (scores[j] < scores[j + 1]) {
            int temp = scores[j];
            scores[j] = scores[j + 1];
            scores[j + 1] = temp;
        }
    }
}

// ◊ Display Top Performers (Top 10)
System.out.println("\nTop Placement-Eligible Students:");
int limit = Math.min(10, n);

for (int i = 0; i < limit; i++) {
    System.out.println("Rank " + (i + 1) + " : " + scores[i]);
}

sc.close();
}
}

```

Rank learners on an online platform by sorting quiz scores using Selection Sort and display top 10 learners.

```

import java.util.*;

class OnlineLeaderboard {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of learners: ");
        int n = sc.nextInt();

        int[] scores = new int[n];

        System.out.println("Enter quiz scores:");
        for (int i = 0; i < n; i++) {
            scores[i] = sc.nextInt();
        }
    }
}

```

```

// ◊ Selection Sort (Descending order)
for (int i = 0; i < n - 1; i++) {
    int maxIndex = i;

    for (int j = i + 1; j < n; j++) {
        if (scores[j] > scores[maxIndex]) {
            maxIndex = j;
        }
    }

    // swap
    int temp = scores[i];
    scores[i] = scores[maxIndex];
    scores[maxIndex] = temp;
}

// ◊ Display Top 10 Learners
System.out.println("\nTop 10 Learners:");
int limit = Math.min(10, n);

for (int i = 0; i < limit; i++) {
    System.out.println("Rank " + (i + 1) + " : " + scores[i]);
}

sc.close();
}
}

```

Select top athletes by sorting performance scores using Insertion Sort and display top 10 players.

```

import java.util.*;

class AthleteRanking {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of athletes: ");
        int n = sc.nextInt();

        int[] scores = new int[n];

        System.out.println("Enter performance scores:");
        for (int i = 0; i < n; i++) {
            scores[i] = sc.nextInt();
        }
    }
}

```

```

// ◊ Insertion Sort (Descending order)
for (int i = 1; i < n; i++) {
    int key = scores[i];
    int j = i - 1;

    while (j >= 0 && scores[j] < key) {
        scores[j + 1] = scores[j];
        j--;
    }

    scores[j + 1] = key;
}

// ◊ Display Top 10 Players
System.out.println("\nTop 10 Athletes:");
int limit = Math.min(10, n);

for (int i = 0; i < limit; i++) {
    System.out.println("Rank " + (i + 1) + " : " + scores[i]);
}

sc.close();
}
}

```

Arrange library books by sorting accession numbers using Insertion Sort and display the ordered list.

```

import java.util.*;

class LibraryBooks {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of books: ");
        int n = sc.nextInt();

        int[] accNo = new int[n];

        System.out.println("Enter accession numbers:");
        for (int i = 0; i < n; i++) {
            accNo[i] = sc.nextInt();
        }

        // ◊ Insertion Sort (Ascending order)
        for (int i = 1; i < n; i++) {
            int key = accNo[i];
            int j = i - 1;

```

```

        while (j >= 0 && accNo[j] > key) {
            accNo[j + 1] = accNo[j];
            j--;
        }

        accNo[j + 1] = key;
    }

    // ◊ Display sorted list
    System.out.println("\nSorted Accession Numbers:");
    for (int num : accNo) {
        System.out.print(num + " ");
    }

    sc.close();
}
}

```

Find the greatest common divisor (GCD) of two numbers to simplify ratios.

```

import java.util.*;

class GCDExample {

    // Function to find GCD
    static int gcd(int a, int b) {
        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }
        return a;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter two numbers: ");
        int a = sc.nextInt();
        int b = sc.nextInt();

        int gcd = gcd(a, b);

        System.out.println("GCD = " + gcd);

        // Simplified ratio
        System.out.println("Simplified Ratio = " + (a / gcd) + " : " + (b / gcd));
    }
}

```

```

        sc.close();
    }
}

```

Generate Fibonacci sequence to model progressive growth patterns.

```

import java.util.*;

class FibonacciExample {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of terms: ");
        int n = sc.nextInt();

        int a = 0, b = 1;

        System.out.println("Fibonacci Sequence:");

        for (int i = 0; i < n; i++) {
            System.out.print(a + " ");

            int next = a + b;
            a = b;
            b = next;
        }

        sc.close();
    }
}

```

Search for a specific element in an unsorted list using sequential search.

```

import java.util.*;

class SequentialSearch {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();

        int[] arr = new int[n];
    }
}

```



```

        System.out.println("Enter elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.print("Enter element to search: ");
        int key = sc.nextInt();

        boolean found = false;

        // ◊ Sequential Search
        for (int i = 0; i < n; i++) {
            if (arr[i] == key) {
                System.out.println("Element found at position: " + (i + 1));
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("Element not found");
        }

        sc.close();
    }
}

```

Locate a target element in a sorted array using binary search.

```

import java.util.*;

class BinarySearchExample {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();

        int[] arr = new int[n];

        System.out.println("Enter sorted elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.print("Enter element to search: ");
        int key = sc.nextInt();
    }
}

```

```

int low = 0, high = n - 1;
boolean found = false;

// ◊ Binary Search
while (low <= high) {
    int mid = (low + high) / 2;

    if (arr[mid] == key) {
        System.out.println("Element found at position: " + (mid + 1));
        found = true;
        break;
    } else if (key < arr[mid]) {
        high = mid - 1;
    } else {
        low = mid + 1;
    }
}

if (!found) {
    System.out.println("Element not found");
}

sc.close();
}
}

```

Sort a list of elements efficiently using merge sort.

```

class MergeSort {

    void merge(int arr[], int l, int m, int r) {

        int n1 = m - l + 1;
        int n2 = r - m;

        int L[] = new int[n1];
        int R[] = new int[n2];

        for (int i = 0; i < n1; i++)
            L[i] = arr[l + i];

        for (int j = 0; j < n2; j++)
            R[j] = arr[m + 1 + j];

        int i = 0, j = 0, k = l;

        while (i < n1 && j < n2) {
            if (L[i] <= R[j])

```

```

        arr[k++] = L[i++];
    else
        arr[k++] = R[j++];
}

while (i < n1)
    arr[k++] = L[i++];

while (j < n2)
    arr[k++] = R[j++];
}

void sort(int arr[], int l, int r) {
    if (l < r) {

        int m = (l + r) / 2;

        sort(arr, l, m);
        sort(arr, m + 1, r);

        merge(arr, l, m, r);
    }
}

public static void main(String[] args) {

    int arr[] = {5, 8, 2, 3, 1};

    MergeSort obj = new MergeSort();
    obj.sort(arr, 0, arr.length - 1);

    System.out.println("Sorted array:");

    for (int num : arr)
        System.out.print(num + " ");
}
}

```

Arrange elements based on priority using heap sort.

```

class HeapSort {

    void heapify(int arr[], int n, int i) {
        int largest = i;
        int left = 2*i + 1;
        int right = 2*i + 2;

        if (left < n && arr[left] > arr[largest])

```

```

        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void sort(int arr[]) {
    int n = arr.length;

    // ◊ Build Max Heap
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // ◊ Extract elements
    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

public static void main(String[] args) {

    int arr[] = {5, 8, 2, 3, 1};

    HeapSort obj = new HeapSort();
    obj.sort(arr);

    System.out.println("Sorted array (Priority order):");

    for (int num : arr)
        System.out.print(num + " ");
}

```

. Sort a dataset quickly using quick sort algorithm.

```

class QuickSort {

```

```

int partition(int arr[], int low, int high) {
    int pivot = arr[high];    // last element as pivot
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;

            // swap
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    // place pivot correctly
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void sort(int arr[], int low, int high) {
    if (low < high) {

        int pi = partition(arr, low, high);

        sort(arr, low, pi - 1);    // left side
        sort(arr, pi + 1, high);    // right side
    }
}

public static void main(String[] args) {

    int arr[] = {5, 8, 2, 3, 1};

    QuickSort obj = new QuickSort();
    obj.sort(arr, 0, arr.length - 1);

    System.out.println("Sorted array:");

    for (int num : arr)
        System.out.print(num + " ");
}
}

```

Improve sorting efficiency using randomized quick sort.

```

import java.util.Random;

class RandomQuickSort {

    int partition(int arr[], int low, int high) {
        int pivot = arr[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;

                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;

        return i + 1;
    }

    void randomSort(int arr[], int low, int high) {
        if (low < high) {

            Random rand = new Random();

            // 🎲 Random pivot selection
            int pivotIndex = low + rand.nextInt(high - low + 1);

            // swap random pivot with last
            int temp = arr[pivotIndex];
            arr[pivotIndex] = arr[high];
            arr[high] = temp;

            int pi = partition(arr, low, high);

            randomSort(arr, low, pi - 1);
            randomSort(arr, pi + 1, high);
        }
    }

    public static void main(String[] args) {

        int arr[] = {5, 8, 2, 3, 1};
    }
}

```

```

RandomQuickSort obj = new RandomQuickSort();
obj.randomSort(arr, 0, arr.length - 1);

System.out.println("Sorted array:");

for (int num : arr)
    System.out.print(num + " ");
}
}

```

Maximize profit by selecting items within capacity using fractional knapsack.

```

import java.util.*;

class FractionalKnapsack {

    public static void main(String[] args) {

        int[] weight = {10, 20, 30};
        int[] value = {60, 100, 120};
        int capacity = 50;

        int n = weight.length;

        double[][] ratio = new double[n][2];

        // store index and ratio
        for (int i = 0; i < n; i++) {
            ratio[i][0] = i;
            ratio[i][1] = (double) value[i] / weight[i];
        }

        // sort by ratio descending
        Arrays.sort(ratio, (a, b) -> Double.compare(b[1], a[1]));

        double maxProfit = 0.0;

        for (int i = 0; i < n; i++) {
            int index = (int) ratio[i][0];

            if (capacity >= weight[index]) {
                maxProfit += value[index];
                capacity -= weight[index];
            } else {
                maxProfit += ratio[i][1] * capacity;
                break;
            }
        }
    }
}

```

```

    }

    System.out.println("Maximum Profit: " + maxProfit);
}
}

```

Schedule jobs to maximize profit within deadlines using job sequencing.

```

import java.util.*;

class Job {
    int id, deadline, profit;

    Job(int id, int d, int p) {
        this.id = id;
        this.deadline = d;
        this.profit = p;
    }
}

class JobSequencing {
    public static void main(String[] args) {

        Job[] jobs = {
            new Job(1, 2, 100),
            new Job(2, 1, 50),
            new Job(3, 2, 10),
            new Job(4, 1, 20)
        };

        int n = jobs.length;

        // ◊ Sort jobs by profit (descending)
        Arrays.sort(jobs, (a, b) -> b.profit - a.profit);

        int maxDeadline = 0;
        for (Job job : jobs)
            maxDeadline = Math.max(maxDeadline, job.deadline);

        int[] slot = new int[maxDeadline];
        Arrays.fill(slot, -1);

        int totalProfit = 0;

        // ◊ Assign jobs
        for (int i = 0; i < n; i++) {
            for (int j = jobs[i].deadline - 1; j >= 0; j--) {

```



```

        if (slot[j] == -1) {
            slot[j] = jobs[i].id;
            totalProfit += jobs[i].profit;
            break;
        }
    }
}

// ◊ Output
System.out.println("Selected Jobs:");
for (int i = 0; i < maxDeadline; i++) {
    if (slot[i] != -1)
        System.out.print("J" + slot[i] + " ");
}

System.out.println("\nTotal Profit: " + totalProfit);
}
}

```

Construct a minimum spanning tree using Kruskal's algorithm.

```

import java.util.*;

class Edge {
    int src, dest, weight;

    Edge(int s, int d, int w) {
        src = s;
        dest = d;
        weight = w;
    }
}

class KruskalMST {

    static int find(int parent[], int i) {
        if (parent[i] == i)
            return i;
        return find(parent, parent[i]);
    }

    static void union(int parent[], int x, int y) {
        int xset = find(parent, x);
        int yset = find(parent, y);
        parent[xset] = yset;
    }

    public static void main(String[] args) {

```

```

int V = 4; // vertices
Edge[] edges = {
    new Edge(0, 1, 1),
    new Edge(0, 2, 3),
    new Edge(1, 2, 2),
    new Edge(1, 3, 4),
    new Edge(2, 3, 5)
};

// ◊ Sort edges by weight
Arrays.sort(edges, (a, b) -> a.weight - b.weight);

int parent[] = new int[V];
for (int i = 0; i < V; i++)
    parent[i] = i;

System.out.println("Edges in MST:");

int count = 0, i = 0;
int totalWeight = 0;

while (count < V - 1) {
    Edge next = edges[i++];

    int x = find(parent, next.src);
    int y = find(parent, next.dest);

    if (x != y) {
        System.out.println(next.src + " - " + next.dest + " : " +
next.weight);
        totalWeight += next.weight;
        union(parent, x, y);
        count++;
    }
}

System.out.println("Total Weight: " + totalWeight);
}
}

```

Build a minimum cost spanning tree using Prim's algorithm.

```

import java.util.*;

class PrimsMST {

    int minKey(int key[], boolean mstSet[], int V) {
        int min = Integer.MAX_VALUE, minIndex = -1;

```

```

    for (int v = 0; v < V; v++) {
        if (!mstSet[v] && key[v] < min) {
            min = key[v];
            minIndex = v;
        }
    }
    return minIndex;
}

void primMST(int graph[][], int V) {

    int parent[] = new int[V];    // MST
    int key[] = new int[V];      // min weights
    boolean mstSet[] = new boolean[V];

    Arrays.fill(key, Integer.MAX_VALUE);
    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < V - 1; count++) {

        int u = minKey(key, mstSet, V);
        mstSet[u] = true;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] != 0 && !mstSet[v] && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    // Print MST
    System.out.println("Edges in MST:");
    int totalCost = 0;

    for (int i = 1; i < V; i++) {
        System.out.println(parent[i] + " - " + i + " : " + graph[i][parent[i]]);
        totalCost += graph[i][parent[i]];
    }

    System.out.println("Total Cost: " + totalCost);
}

public static void main(String[] args) {

    int graph[][] = {
        {0, 2, 0, 6},
        {2, 0, 3, 8},

```

```

        {0, 3, 0, 0},
        {6, 8, 0, 0}
    };

    int V = 4;

    PrimsMST obj = new PrimsMST();
    obj.primMST(graph, V);
}
}

```

Find the shortest path from a source node using Dijkstra's algorithm.

```

import java.util.*;

class Dijkstra {

    int minDistance(int dist[], boolean visited[], int V) {
        int min = Integer.MAX_VALUE, minIndex = -1;

        for (int i = 0; i < V; i++) {
            if (!visited[i] && dist[i] < min) {
                min = dist[i];
                minIndex = i;
            }
        }
        return minIndex;
    }

    void dijkstra(int graph[][], int src, int V) {

        int dist[] = new int[V];
        boolean visited[] = new boolean[V];

        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        for (int count = 0; count < V - 1; count++) {

            int u = minDistance(dist, visited, V);
            visited[u] = true;

            for (int v = 0; v < V; v++) {
                if (!visited[v] && graph[u][v] != 0 &&
                    dist[u] != Integer.MAX_VALUE &&
                    dist[u] + graph[u][v] < dist[v]) {

                    dist[v] = dist[u] + graph[u][v];
                }
            }
        }
    }
}

```

```

        }
    }
}

// Print result
System.out.println("Shortest distances from source:");
for (int i = 0; i < V; i++) {
    System.out.println("Node " + i + " : " + dist[i]);
}
}

public static void main(String[] args) {

    int graph[][] = {
        {0, 4, 1, 0},
        {4, 0, 2, 5},
        {1, 2, 0, 8},
        {0, 5, 8, 0}
    };

    int V = 4;

    Dijkstra obj = new Dijkstra();
    obj.dijkstra(graph, 0, V);
}
}

```

Solve the N-Queens problem using dynamic programming concepts to place N queens on an NxN chessboard such that no two queens attack each other.

```

import java.util.*;

class NQueensDP {

    int N;
    boolean[] cols, diag1, diag2;

    void solve(int row, int[][] board) {

        if (row == N) {
            printBoard(board);
            return;
        }

        for (int col = 0; col < N; col++) {

            if (cols[col] || diag1[row - col + N - 1] || diag2[row + col])
                continue;

```

```

        // place queen
        board[row][col] = 1;
        cols[col] = diag1[row - col + N - 1] = diag2[row + col] = true;

        solve(row + 1, board);

        // backtrack
        board[row][col] = 0;
        cols[col] = diag1[row - col + N - 1] = diag2[row + col] = false;
    }
}

void start(int n) {
    N = n;
    int[][] board = new int[N][N];

    cols = new boolean[N];
    diag1 = new boolean[2 * N];
    diag2 = new boolean[2 * N];

    solve(0, board);
}

void printBoard(int[][] board) {
    System.out.println("Solution:");
    for (int[] row : board) {
        for (int val : row)
            System.out.print(val + " ");
        System.out.println();
    }
    System.out.println();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter N: ");
    int n = sc.nextInt();

    NQueensDP obj = new NQueensDP();
    obj.start(n);

    sc.close();
}
}

```

Implement a solution for the 8-Queens problem to place 8 queens on a chessboard without conflicts using an optimized approach.

```

class EightQueens {

    static final int N = 8;

    boolean[] cols = new boolean[N];
    boolean[] diag1 = new boolean[2 * N];
    boolean[] diag2 = new boolean[2 * N];

    int[][] board = new int[N][N];

    void solve(int row) {

        if (row == N) {
            printBoard();
            return;
        }

        for (int col = 0; col < N; col++) {

            // check if safe
            if (cols[col] || diag1[row - col + N - 1] || diag2[row + col])
                continue;

            // place queen
            board[row][col] = 1;
            cols[col] = diag1[row - col + N - 1] = diag2[row + col] = true;

            solve(row + 1);

            // backtrack
            board[row][col] = 0;
            cols[col] = diag1[row - col + N - 1] = diag2[row + col] = false;
        }
    }

    void printBoard() {
        System.out.println("Solution:");
        for (int[] row : board) {
            for (int val : row)
                System.out.print(val + " ");
            System.out.println();
        }
        System.out.println();
    }

    public static void main(String[] args) {
        EightQueens obj = new EightQueens();
        obj.solve(0);
    }
}

```

```
    }  
}
```

Place 4 queens on a 4x4 chessboard such that no two queens attack each other using an efficient algorithmic approach.

```
class FourQueens {  
  
    static final int N = 4;  
  
    boolean[] cols = new boolean[N];  
    boolean[] diag1 = new boolean[2 * N];  
    boolean[] diag2 = new boolean[2 * N];  
  
    int[][] board = new int[N][N];  
  
    void solve(int row) {  
  
        if (row == N) {  
            printBoard();  
            return;  
        }  
  
        for (int col = 0; col < N; col++) {  
  
            // check safe  
            if (cols[col] || diag1[row - col + N - 1] || diag2[row + col])  
                continue;  
  
            // place queen  
            board[row][col] = 1;  
            cols[col] = diag1[row - col + N - 1] = diag2[row + col] = true;  
  
            solve(row + 1);  
  
            // backtrack  
            board[row][col] = 0;  
            cols[col] = diag1[row - col + N - 1] = diag2[row + col] = false;  
        }  
    }  
  
    void printBoard() {  
        System.out.println("Solution:");  
        for (int[] row : board) {  
            for (int val : row)  
                System.out.print(val + " ");  
            System.out.println();  
        }  
        System.out.println();  
    }  
}
```



```
    }  
    public static void main(String[] args) {  
        FourQueens obj = new FourQueens();  
        obj.solve(0);  
    }  
}
```